

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGENO
1	Abstract	1
	Introduction	
2	System Study	3
	2.1 Existing system	
	2.1.1 Drawbacks	
	2.2 Proposed System	
	2.2.1 Advantages	
	2.3 Feasibility study	
	2.3.1 Legal and Ethical feasibility	
	2.3.2 Technical feasibility	
3	System Specification	7
	3.1 Hardware Specification	
	3.2 Software Specification	
4	Software Description	8
	4.1 Software Requirements	
5	Project Description	14
	5.1 Overview of the project	
	5.2 Modules	
	5.2.1 User Management Module	
	5.2.2 Product Management Module	
	5.2.3 Order Management Module	
	5.2.4 Payment Management Module	
	5.2.5 Admin Management Module	
	5.2.6 Result & Status Module	
	5.3 Data Flow Diagram	
	5.4 System Design	

6	System Testing	19
	6.1 Unit Testing	
	6.2 Integration Testing	
	6.3 System Testing Scenarios	
	6.4 Performance Testing	
	6.5 User Acceptance Testing	
	6.6 Test case	
7	System Implementation	23
8	Future Enhancement	24
	& Conclusion	
	8.1 Conclusion	
	8.2 Future Enhancement	
9	Appendix	26
	9.1 Screenshots	
	9.2 Source Code	

CHAPTER 1

ABSTRACT

Smart farmer marketplaces have gained increasing importance due to the growing demand for fresh agricultural products, direct farmer-to-consumer trade, and the adoption of digital technologies in agriculture. However, traditional agricultural marketing systems largely depend on intermediaries, manual processes, and limited market access, which often result in unfair pricing, reduced farmer profits, and lack of transparency for buyers. Farmers face challenges in reaching a wider customer base, promoting their products digitally, and managing orders efficiently. Buyers, on the other hand, struggle to access reliable information about product availability, quality, pricing, and origin. To address these challenges, this project presents a Smart Farmer Marketplace and E-Digital Marketing System that provides a digital, efficient, and farmer-centric trading platform. The proposed system enables farmers to list agricultural products with detailed information such as category, price, quantity, and images, while buyers can browse products, place orders, and provide feedback through an online interface. The system manages product availability, order processing, and promotional activities to ensure smooth transactions and prevent conflicts. Role-based access control ensures secure and independent workflows for farmers, buyers, and administrators, allowing effective management of users, products, and orders. The backend of the system is developed using Python Flask, while the frontend is built using HTML, CSS, JavaScript, and Bootstrap to provide a responsive and user-friendly experience. SQLite is used for reliable data storage and management. By digitizing agricultural trade and marketing, the proposed system enhances operational efficiency, improves market reach for farmers, and delivers a scalable solution for modern agricultural commerce.

INTRODUCTION

Traditional agricultural marketing systems rely heavily on local markets, intermediaries, and manual trading practices, which are often inefficient, time-consuming, and lacking transparency. These approaches struggle to handle increasing consumer demand, multiple product categories, and fluctuating availability, especially in regions where farmers have limited access to digital platforms. As a result, farmers face challenges such as reduced profit margins, limited market reach, and difficulty in promoting products, while buyers encounter issues like unclear pricing, lack of product information, and limited availability. To overcome these limitations, digital marketplace solutions have emerged as effective approaches for automating agricultural trade and improving accessibility for both farmers and consumers. This project aims to develop a Smart Farmer Marketplace and E-Digital Marketing System that provides an automated and user-friendly platform for listing, browsing, and purchasing agricultural products online. The proposed system enables farmers to upload product details, manage inventory, and promote products digitally, while buyers can search for products, place orders, and receive confirmations through a structured and transparent process. In addition, the system supports centralized management of users, products, and transactions, allowing administrators to efficiently monitor platform activities. Role-based access ensures secure and organized workflows for farmers, buyers, and administrators. The backend of the application is developed using Python Flask to manage business logic and data operations, while the frontend is built using HTML, CSS, JavaScript, and Bootstrap to provide an interactive and responsive user interface for efficient agricultural commerce.

CHAPTER 2

SYSTEM STUDY

2.1 EXISTING SYSTEM

- ❖ In the existing system, agricultural product trading is mainly handled through traditional methods such as local markets, middlemen, and manual record keeping for sales and inventory.
- ❖ Traditional agricultural marketing systems require farmers to physically visit markets to sell their produce, while buyers depend on intermediaries to access products. Product availability, pricing, and transactions are managed manually, making the process time-consuming, inefficient, and lacking real-time coordination between farmers and buyers.

2.1.1 DRAWBACKS

The existing system has following disadvantages,

- ❖ Manual trading processes result in delays and inconvenience for both farmers and buyers.
- ❖ Farmers depend heavily on intermediaries, reducing profit margins and increasing operational complexity.
- ❖ Buyers have limited access to real-time information about product availability, pricing, and quality.
- ❖ Traditional systems are not efficient in handling large volumes of products and multiple buyers simultaneously.
- ❖ Lack of digital integration leads to poor record management, limited transparency, and difficulty in tracking sales history and inventory levels.

2.2 PROPOSED SYSTEM

- ❖ The proposed system introduces a digital and automated solution for managing agricultural product sales through an online marketplace. Instead of relying on traditional market visits and manual trading practices, the system enables farmers and buyers to list, browse, and manage agricultural transactions using a web-based application developed with modern web technologies.
- ❖ The system collects farmer product details, pricing, quantity, and buyer order information through an interactive web interface and processes the data using backend logic implemented in Python Flask. Based on selected products and order quantities, the system automatically manages inventory levels, order status, and transaction records to ensure accurate and organized order processing.
- ❖ A validation mechanism is applied to prevent order conflicts and manage stock availability effectively by checking product quantity before confirming orders. Role-based access control ensures that farmers, buyers, and administrators have clearly defined permissions and responsibilities within the system, enabling secure and structured platform operation.
- ❖ The system supports centralized management of users, products, orders, and promotional activities, allowing administrators to approve products, monitor transactions, and manage platform operations efficiently. All order details, product information, and user records are securely stored in the database for tracking and future reference.
- ❖ This approach provides an efficient, scalable, and user-friendly alternative to traditional agricultural marketing systems. The proposed system reduces manual effort, improves farmer and buyer convenience, and enhances overall agricultural trade management. It can be easily adopted by farming communities and supports future enhancements such as online payment integration, mobile application support, and advanced analytics for demand and pricing trends.

.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

- ❖ Fully automated system that enables online product listing, browsing, ordering, and sales management without continuous manual intervention.
- ❖ Allows buyers to purchase agricultural products based on real-time availability, pricing, and quantity details.
- ❖ Stores farmer information, buyer details, order history, feedback, and transaction records securely for future reference and analysis.
- ❖ Supports farmers and administrators in managing products, inventory, orders, and users efficiently while reducing administrative workload.
- ❖ Helps farmers and buyers save time by eliminating the need for physical market visits and minimizing delays in transactions.
- ❖ Cost-effective solution developed using open-source technologies and lightweight tools.
- ❖ Ensures organized and conflict-free order processing through automated inventory checks and validation mechanisms.
- ❖ Easy to use with a simple, intuitive, and user-friendly web interface.
- ❖ Reliable system performance with accurate order management and secure role-based access for farmers, buyers, and administrators.
- ❖ Scalable architecture capable of handling a large number of products, users, and simultaneous transactions.
- ❖ Capable of integrating additional features such as online payments, notifications, mobile applications, and advanced analytics in the future.

2.3 FEASIBILITY STUDY

2.3.1 LEGAL AND ETHICAL FEASIBILITY

LEGAL FEASIBILITY

- The system collects user information such as name, email, role (farmer, buyer, admin), and transaction details only with the user's consent during registration and platform usage.
- No sensitive personal data such as banking credentials, government IDs, or biometric information is stored, ensuring compliance with basic data privacy and security guidelines. The implementation does not violate any legal or regulatory standards, as it is developed as an academic project focused on digital agricultural product trading and marketing.

ETHICAL FEASIBILITY

- The system ensures user data privacy by securely storing information and controlling access through role-based authentication, providing fair and transparent access to the marketplace for all users. It promotes ethical and socially responsible practices by enabling direct farmer-to-buyer interactions, reducing intermediaries, and supporting transparent and sustainable agricultural trade through digital technology.

2.3.2 TECHNICAL FEASIBILITY

HARDWARE FEASIBILITY

- Requires only a standard personal computer or laptop with basic processing capabilities. No specialized hardware or external devices are required to run the system. The system can be accessed using common devices such as desktops, laptops, or mobile phones with a web browser and local network or internet connectivity.

SOFTWARE FEASIBILITY

- Implemented using Python and Flask, which are free, open-source, and widely supported technologies. Uses SQLite for database management, which is lightweight and suitable for small to medium-scale smart farmer marketplace applications.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 HARDWARE SPECIFICATION

This section gives the details and specification of the hardware on which the system is expected to work.

- Processor : Intel Core i3.
- RAM : 4 GB of RAM or more.
- Hard Disk : 360 GB of disk space.
- Keyboard : Standard keyboard.
- Monitor : 1280x720 resolution or higher recommended.

3.2 SOFTWARE CONFIGURATION

This section gives the details of the software that are used for the development.

- Coding Language : Python 3.8
- Libraries Used : Flask, SQLAlchemy, SQLite3, Werkzeug Security Utilities
- Operating System : Windows®11.

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 SOFTWARE REQUIREMENTS

PYTHON

Python provides a powerful and flexible environment for developing web-based agricultural marketplace applications. It supports rapid development, clean syntax, and seamless integration with databases and web frameworks. Its extensive ecosystem and strong community support make it suitable for building reliable backend systems.

In this project, Python is used for:

- Implementing backend business logic for product listing, order processing, and marketplace management.
- Managing user authentication and role-based access control for admin, farmers, and buyers. Handling inventory validation, order confirmation, and transaction management.
- Integrating backend services with frontend components using Flask. Interacting with the SQLite database through SQLAlchemy to store and retrieve system data.

Python integrates smoothly with web technologies and databases, making it ideal for building a scalable and maintainable smart farmer management system.

FEATURES

- **High-Level Language:** Simple syntax improves development speed and code readability.
- **Cross-Platform Compatibility:** Runs on Windows, Linux, and macOS.
- **Rapid Development:** Enables faster implementation of web applications.
- **Extensive Ecosystem:** Supports a wide range of libraries and frameworks.

FLASK

Flask is a lightweight Python web framework used to build the backend of the Smart Farmer Marketplace system. It manages HTTP requests, application routing, sessions, and connects backend logic with frontend templates.

Key functionalities include:

- Handling user registration, login, and logout for farmers, buyers, and admin.
- Managing role-based dashboards for different user roles.
- Processing product listings, order placement, and transaction workflows.
- Controlling application routing, sessions, and request handling..

Flask ensures modular design and easy maintenance of the application.

FEATURES

- Lightweight Framework: Simple and efficient for academic projects.
- Routing Support: Manages URL endpoints and page navigation.
- Session Management: Supports secure user authentication.
- Template Integration: Works seamlessly with HTML and Bootstrap.
- Scalable Design: Can be extended with additional modules and features.

SQLALCHEMY

SQLAlchemy is an Object Relational Mapping (ORM) tool used to interact with the SQLite database in an efficient and secure manner. It simplifies database operations by allowing developers to work with Python objects instead of raw SQL queries.

In this project, SQLAlchemy is used for:

- Defining database models for users, farmers, buyers, products, orders, and feedback.
- Managing relationships between tables such as farmers and products, and orders and buyers.
- Performing database operations like insert, update, delete, and fetch.
- Ensuring secure and optimized database interaction.

SQLAlchemy improves code readability and reduces database-related errors.

FEATURES

- ORM Support: Maps database tables to Python classes.
- Secure Queries: Helps prevent SQL injection attacks.
- Easy Maintenance: Simplifies database schema management.
- Efficient Data Handling: Optimizes database transactions.
- Flask Integration: Works seamlessly with Flask applications.

SQLITE3 (DATABASE)

SQLite is a lightweight, embedded relational database used to store all system data for the Smart Farmer Marketplace system. It is well suited for academic and small-scale applications.

For example:

- Stores user credentials and role information.
- Maintains product details such as category, price, quantity, and availability.
- Saves order information, transaction status, and order history.
- Stores feedback, ratings, and promotional data.

SQLite provides reliable data storage without requiring a separate database server.

FEATURES

- **Lightweight Database:** No additional server installation required.
- **Easy Integration:** Works smoothly with Python, Flask, and SQLAlchemy.
- **Structured Data Storage:** Manages relational data efficiently.
- **Low Maintenance:** Ideal for localhost-based projects.
- **Portable:** Database stored as a single file.

WERKZEUG SECURITY UTILITIES

Werkzeug provides security-related utilities that help protect user credentials and maintain secure authentication mechanisms within the system.

In this project, Werkzeug is used to:

- Hash user passwords securely before storing them in the database.
- Verify hashed passwords during login authentication.
- Enhance overall application security by preventing plain-text password storage.

This ensures secure handling of user credentials and protects the system from common security threats.

FEATURES

- Password Hashing: Protects user credentials from exposure.
- Secure Authentication: Prevents plain-text password storage.
- Flask Compatibility: Integrates seamlessly with Flask.
- Reliable Security Tools: Widely used and well-tested utilities

CHAPTER 5

PROJECT DESCRIPTION

5.1 OVERVIEW OF THE PROJECT

This project focuses on providing an efficient and user-friendly solution for connecting farmers and buyers through a digital agricultural marketplace. The Smart Farmer Marketplace system automates the process of listing farm products, browsing available items, placing orders, and managing transactions through an online platform. The system allows farmers to upload product details such as fruits and vegetables, set prices and quantities, while buyers can browse products, view details, and place orders in a transparent and organized manner.

By integrating digital product management and order processing features, the system enables farmers to sell products directly to buyers without relying on traditional intermediaries, reducing delays and manual effort. Role-based access control ensures secure and structured workflows for farmers, buyers, and administrators. The system improves market efficiency, enhances farmer income opportunities, and increases customer satisfaction. It can be effectively used by agricultural producers, local markets, and small-scale farming communities to streamline agricultural trade and digital marketing activities.

5.2 MODULES

- USER MANAGEMENT MODULE
- PRODUCT MANAGEMENT MODULE
- ORDER MANAGEMENT MODULE
- PAYMENT MANAGEMENT MODULE
- ADMIN MANAGEMENT MODULE
- RESULT & STATUS DISPLAY MODULE

5.2.1 USER MANAGEMENT MODULE

The User Management Module handles registration, login, and authentication of all system users, including farmers, buyers, and administrators. It ensures secure access through session-based authentication and role-based authorization. User details such as name, email, role, and account status are stored securely in the database.

This module manages user sessions and ensures that each user can access only the functionalities permitted for their role, thereby maintaining system security, privacy, and data integrity.

5.2.2 PRODUCT MANAGEMENT MODULE

The Product Management Module allows farmers to add, update, and manage agricultural products such as fruits and vegetables. Each product includes details such as product name, category, price, quantity, description, and product images.

This module provides real-time visibility of available products to buyers, enabling them to browse, search, and filter items easily based on category and pricing, supporting informed purchase decisions.

5.2.3 ORDER MANAGEMENT MODULE

The Order Management Module enables buyers to place orders by selecting products and required quantities. The system automatically validates product availability and calculates the total order cost based on selected items and prices.

This module ensures accurate order processing, prevents quantity conflicts, and updates inventory levels after successful order placement, maintaining consistency and reliability in transactions.

5.2.4 PAYMENT MANAGEMENT MODULE

The Payment Management Module handles the processing and tracking of payments for confirmed orders. Buyers can complete payment after placing an order, and the system securely records transaction and payment status in the database.

This module ensures transparent payment handling and supports future integration of online payment gateways and digital wallets.

5.2.5 ADMIN MANAGEMENT MODULE

The Admin Management Module allows administrators to manage the overall marketplace system. Admins can manage user accounts, approve or remove product listings, monitor orders, and oversee marketplace activities.

This module ensures smooth system operation, proper content moderation, and centralized control over farmers, buyers, and product data.

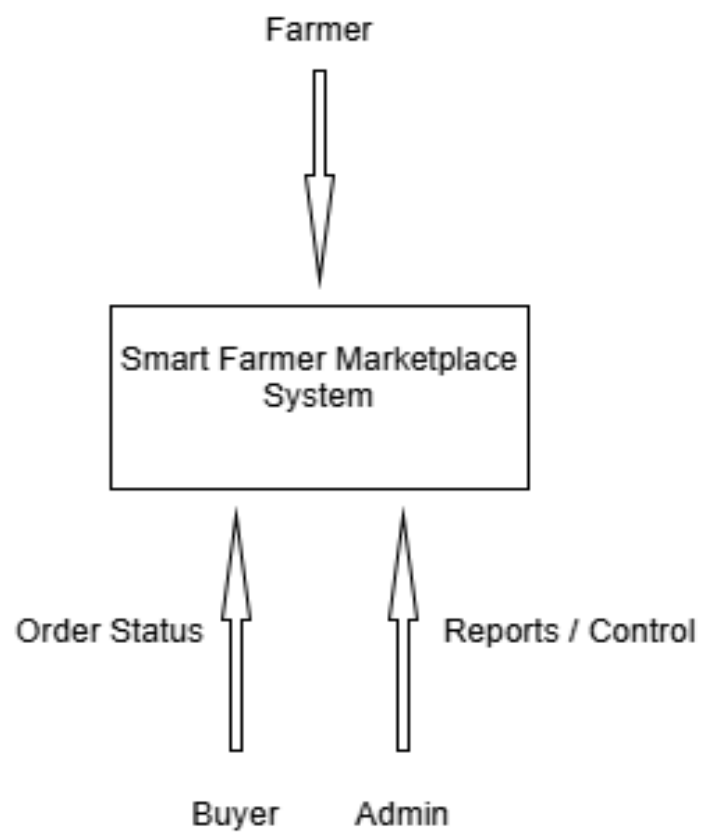
5.2.6 RESULT & STATUS DISPLAY MODULE

The Result and Status Display Module presents order and product information in a clear and user-friendly format. Buyers can view order status, purchase history, and payment details, while farmers can track product sales and order fulfillment.

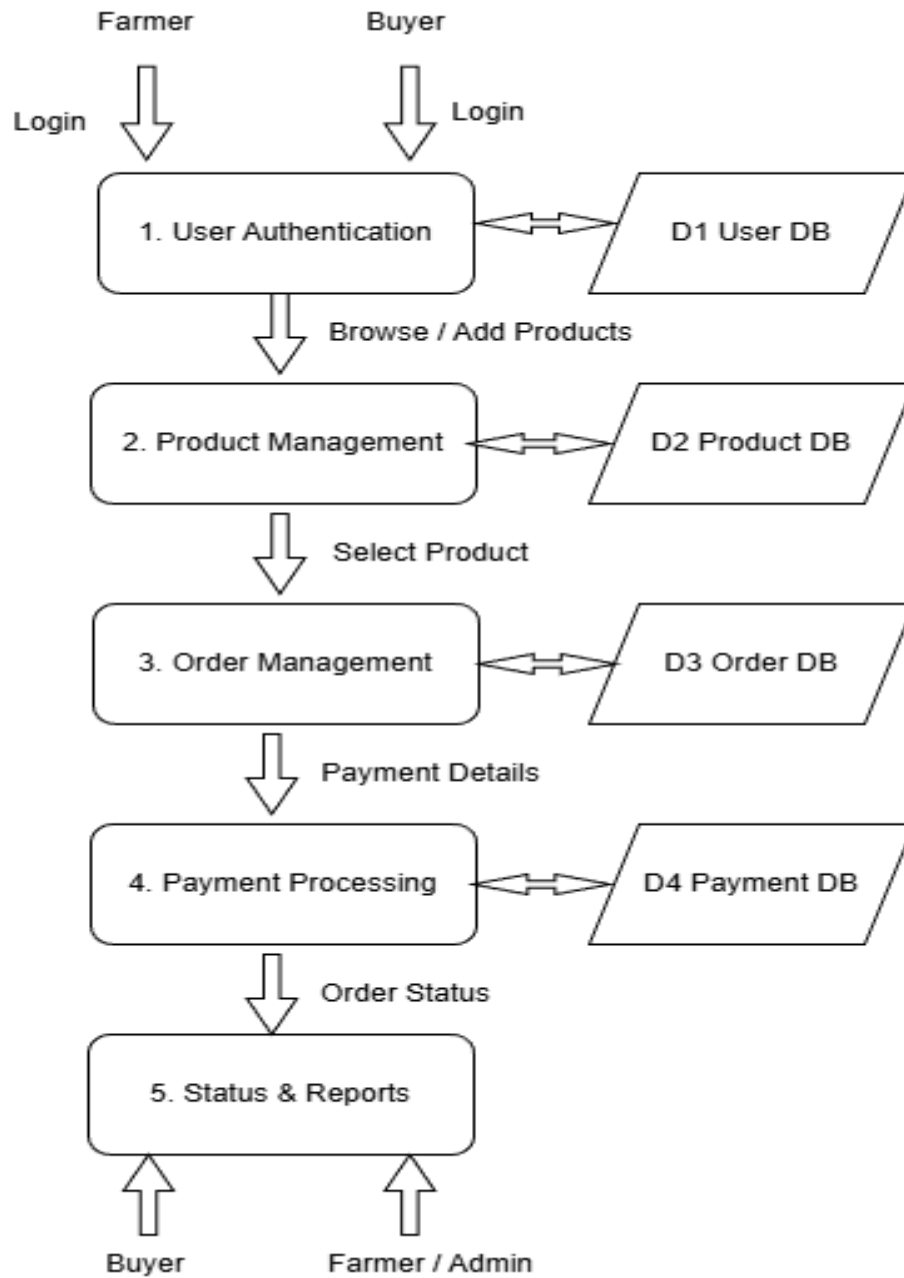
The interface updates dynamically based on system activities, providing real-time visibility into product availability and order status. This module helps users stay informed and improves overall system transparency and usability.

5.3 DATA FLOW DIAGRAM

LEVEL 0:



LEVEL 1: DETAILED DFD



CHAPTER 6

SYSTEM TESTING

6.1 UNIT TESTING

Unit testing is a crucial phase of the software testing process that involves testing individual modules of the Smart Farmer Marketplace independently to ensure that each component functions correctly. In this project, all core modules were tested separately to verify their functionality, reliability, and correctness before system integration.

The User Authentication Module was tested to ensure secure registration and login functionality for farmers, buyers, and administrators. Test cases included invalid login attempts, duplicate registrations, and session handling to confirm proper access control. The Product Management Module was tested to verify that products added by farmers were correctly stored, updated, and displayed. The Order Management Module was tested to ensure that buyers could successfully place orders, select product quantities, and receive order confirmation without errors. Each module was validated to ensure accurate database interaction and error-free operation.

6.2 INTEGRATION TESTING

Integration testing was conducted to verify smooth interaction between the individual modules after unit testing was completed. Modules such as User Authentication, Product Management, Order Management, and Payment Processing were integrated and tested together to ensure proper data flow and communication.

During this phase, user login data was passed to the Product and Order Management modules, where product availability was checked and order details were stored in the database. These order details were then accessed by the Admin Module for monitoring and management.

6.3 SYSTEM TESTING

System testing was performed on the complete Smart Farmer Marketplace system to validate that it meets both functional and non-functional requirements. The system was tested using different user roles such as farmers, buyers, and administrators.

Various scenarios were evaluated, including new user registration, product listing by farmers, order placement during high demand, payment processing, order status updates, and admin operations such as approving products and monitoring transactions. The system was also tested for invalid inputs, missing details, and unauthorized access attempts to ensure stability, reliability, and proper error handling under all conditions.

6.4 PERFORMANCE TESTING

The Performance testing was carried out to evaluate the responsiveness, efficiency, and scalability of the Smart Farmer Marketplace system. The primary goal was to ensure that product browsing, order processing, payment handling, and database operations perform efficiently even with multiple concurrent users.

Key performance metrics such as response time, order confirmation speed, database access time, and system load handling were analyzed. The system was tested with multiple simultaneous order requests to ensure that inventory updates and payment processing were completed within acceptable time limits. The system demonstrated stable performance and quick response times, ensuring a smooth user experience.

6.5 USER ACCEPTANCE TESTING

The User Acceptance Testing (UAT) was conducted to confirm that the system meets real-world user expectations. Test users interacted with the system by registering as farmers and buyers, listing agricultural products, browsing available items, placing orders, making payments, and checking order status.

Administrators evaluated the system by managing user accounts, approving product listings, monitoring orders, and reviewing reports. Feedback from users indicated that the system was easy to use, intuitive, and efficient in managing digital agricultural trade. The clear interface and real-time order updates were positively received, confirming the system's suitability for real-world farmer marketplace applications.

6.6 TEST CASE

The Test Case phase focused on validating the system's functionality through structured testing scenarios to ensure accuracy, reliability, and consistency across all modules.

TEST CASE 1 – USER REGISTRATION AND LOGIN

- Objective: Verify secure registration and authentication for farmers, buyers, and admin.
- Test Steps: Register users with valid and invalid credentials, attempt login, and test session handling.
- Expected Result: Users can register and log in successfully with proper validation and secure access control.

TEST CASE 2 – PRODUCT LISTING FUNCTIONALITY

- Objective: Ensure correct product listing and availability management.
- Test Steps: Add products with valid and invalid inputs and verify visibility.
- Expected Result: Products are listed successfully and stored correctly in the database.

TEST CASE 3 – ORDER MANAGEMENT MODULE

- Objective: Validate order placement and inventory updates.
- Test Steps: Place orders with different quantities and check availability updates.
- Expected Result: Orders are processed successfully and inventory is updated accurately.

TEST CASE 4 – PAYMENT PROCESSING MODULE

- Objective: Verify secure and accurate payment handling.
- Test Steps: Perform payments for bookings and verify transaction status.
- Expected Result: Payments are processed successfully and booking confirmation is generated.

TEST CASE 5 – ROLE-BASED ACCESS CONTROL

- Objective: Ensure proper access restrictions for different users.
- Test Steps: Attempt unauthorized access to restricted pages.
- Expected Result: Users can access only authorized features based on their role.

Through these test cases, the core functionalities of the Smart Farmer Marketplace—including user authentication, product management, order processing, payment handling, and role-based access control—were thoroughly tested, ensuring that the system operates efficiently, securely, and reliably in real-world agricultural marketplace scenarios.

CHAPTER 7

SYSTEM IMPLEMENTATION

The system implementation phase converts the conceptual design of the Smart Farmer Marketplace into a fully functional web-based application. Each module is developed, tested, and integrated in a systematic manner to ensure efficient product listing, order management, and smooth interaction between farmers and buyers. The project primarily uses Python Flask due to its simplicity, flexibility, and strong support for web application development. The implementation begins with the User Authentication Module, which manages secure registration and login for farmers, buyers, and administrators using encrypted passwords and session-based authentication. Once authenticated, users are redirected to role-specific dashboards. Farmers interact with the Product Management Module, where they can add, update, and manage agricultural products such as fruits and vegetables, including price, quantity, category, and images. Buyers interact with the Order Management Module, where they can browse available products, select required quantities, and place orders. Order details are validated and stored securely in an SQLite database to ensure accurate inventory tracking and to prevent quantity conflicts. The Admin Management Module allows administrators to monitor users, approve product listings, and manage overall marketplace activity. Payment processing is handled through the Payment Module, which confirms transactions and records payment status. The Status & Display Module presents order details, product availability, sales information, and payment confirmations through a responsive and user-friendly interface built using HTML, CSS, Bootstrap, and JavaScript. This integrated approach ensures smooth operation, real-time updates, and a reliable digital marketplace experience for farmers, buyers, and administrators.

CHAPTER 8

CONCLUSION & FUTURE ENHANCEMENT

8.1 CONCLUSION

The project “Smart Farmer Marketplace” successfully demonstrates how modern web technologies can be effectively utilized to digitize and streamline agricultural product trading. By integrating secure authentication, product management, order processing, payment handling, and role-based access control, the system provides an efficient and user-friendly digital marketplace. The system eliminates traditional dependency on intermediaries and manual trading processes by enabling direct interaction between farmers and buyers through an online platform. It supports multiple user roles, ensures accurate data management, and maintains structured workflows for farmers, buyers, and administrators. The automated product listing, inventory tracking, and order management features enhance transparency, efficiency, and accessibility in agricultural trade. Overall, the system proves to be a reliable, scalable, and practical solution for supporting farmers and improving digital agricultural marketing.

8.2 FUTURE ENHANCEMENT

The current Smart Farmer Marketplace can be further enhanced to make it more advanced, scalable, and suitable for large-scale agricultural commerce. Future enhancements include:

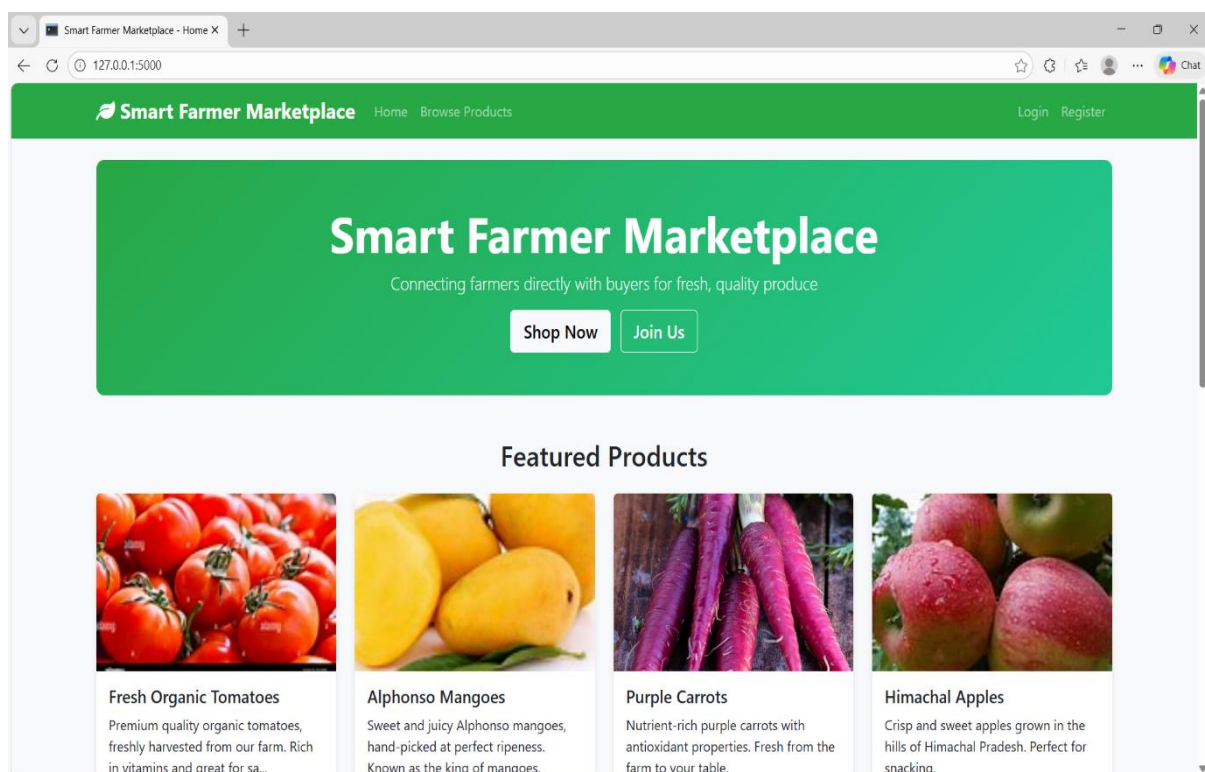
- **Advanced Inventory Management:** Implement real-time stock alerts and automatic quantity updates.
- **Dynamic Pricing System:** Enable demand-based pricing and seasonal price adjustments.
- **Mobile Application Support:** Develop Android and iOS applications for easy farmer and buyer access.
- **Online Payment Gateways:** Integrate secure payment options such as UPI, cards, and digital wallets.
- **Cloud Integration:** Store product data, orders, and analytics securely on cloud platforms.
- **Notification System:** Send SMS or email alerts for order confirmation, delivery updates, and promotions.
- **AI-Based Recommendations:** Suggest products to buyers based on purchase history and demand trends.
- **Multi-Region Support:** Extend the system to support farmers and buyers across multiple regions and markets.

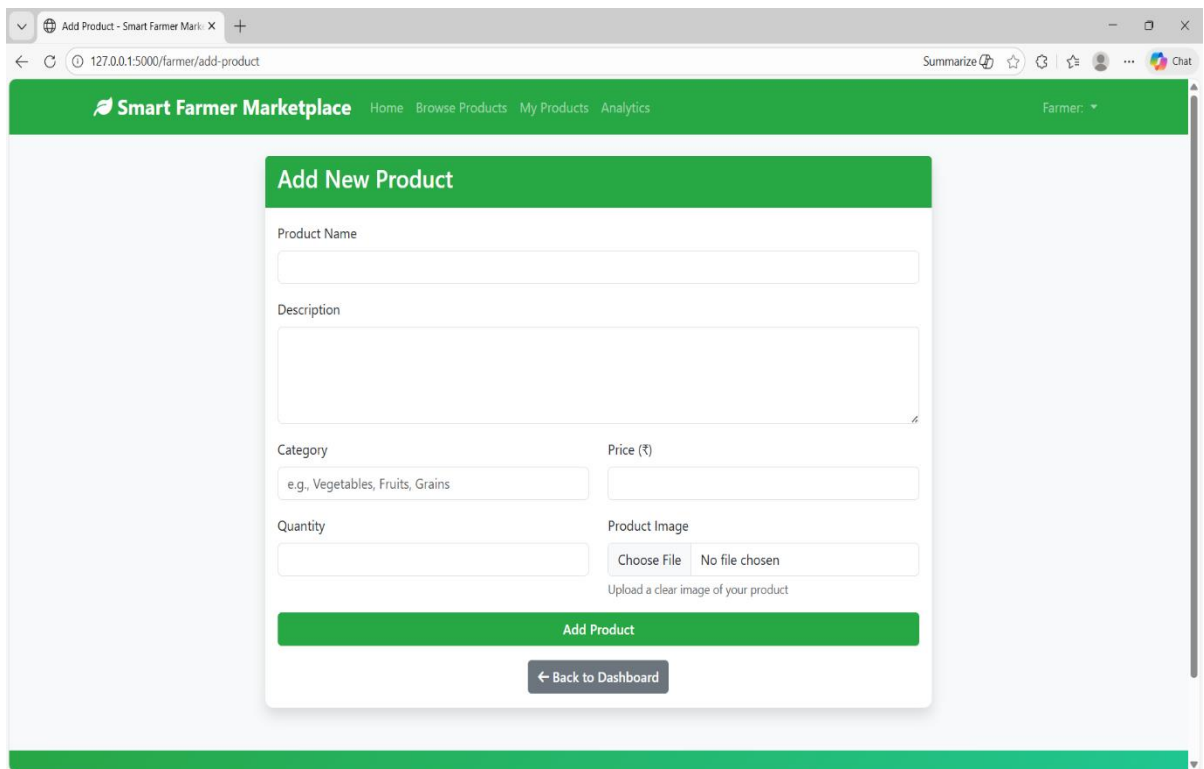
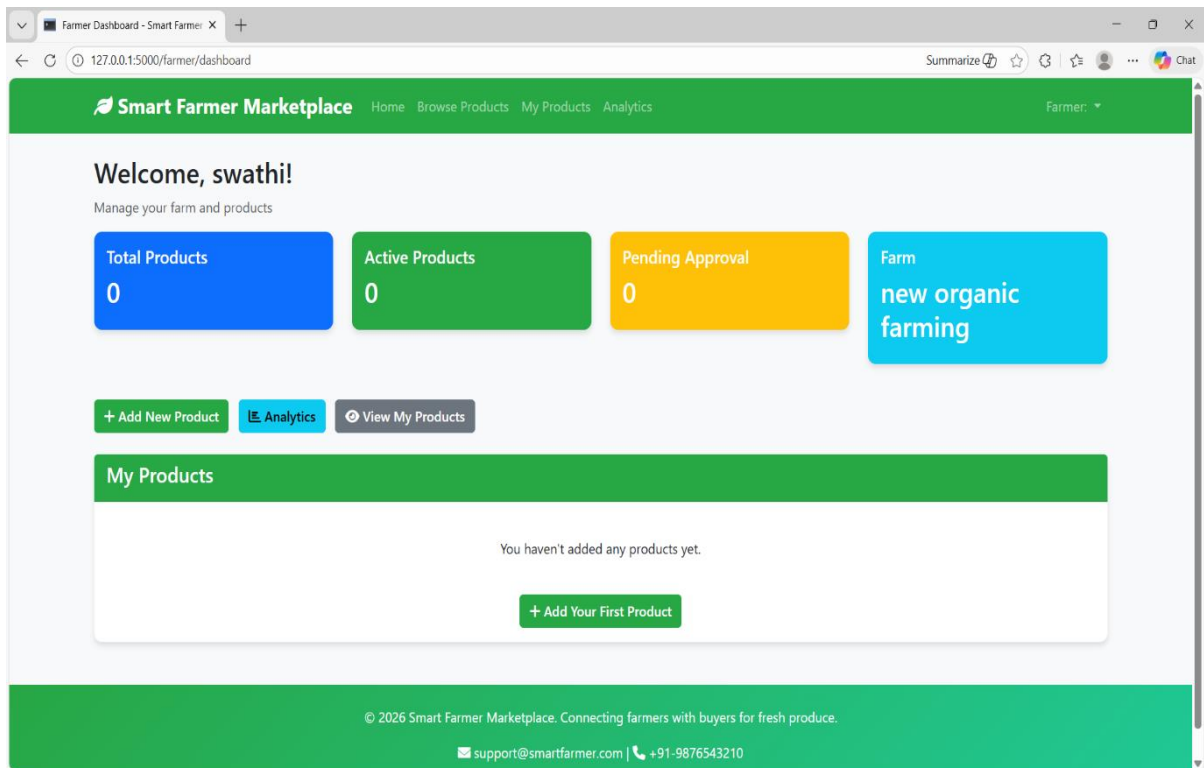
These enhancements will transform the Smart Farmer Marketplace into a robust, intelligent, and scalable digital platform capable of supporting modern agricultural trade and e-digital marketing initiatives.

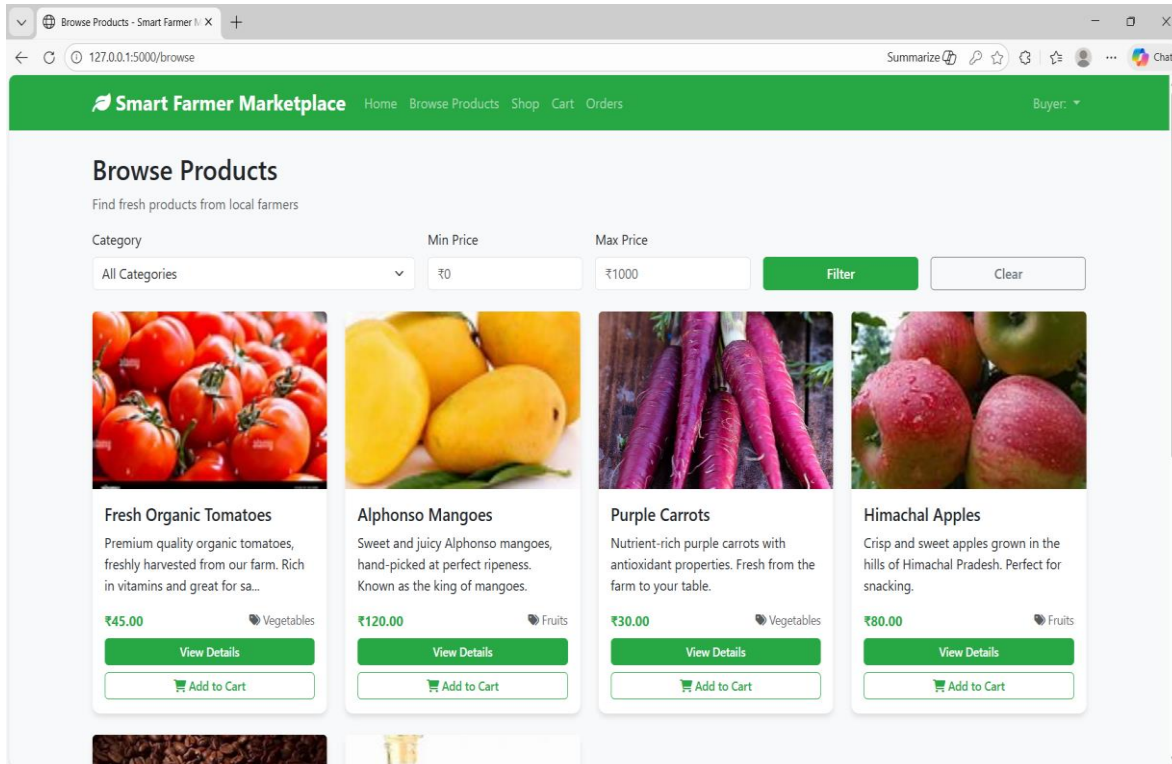
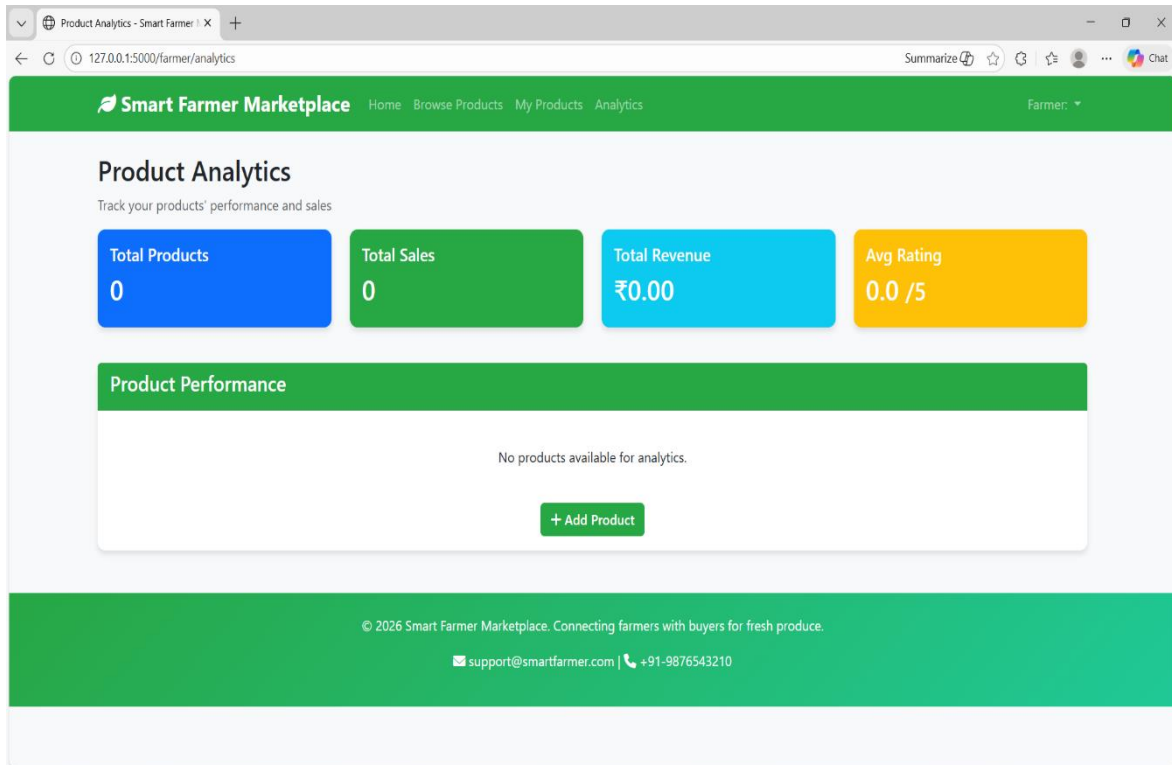
CHAPTER 9

APPENDIX

9.1 SCREENSHOTS







Checkout - Smart Farmer Market

127.0.0.1:5000/checkout

Summarize

Chat

Delivery Information

First Name

Last Name

Email

Phone Number

Delivery Address

City

State

ZIP Code

Payment Method

☐ I agree to the terms and conditions

Place Order

Order Summary

Product	Qty	Subtotal
Alphonso Mangoes	1	₹120.00
Subtotal:		₹120.00
Delivery Fee:		₹50.00
Total:		₹170.00

← Back to Cart

Admin Dashboard - Smart Farmer

127.0.0.1:5000/admin/dashboard

Summarize

Chat

Smart Farmer Marketplace

HomeBrowse ProductsDashboardManage UsersProducts

Admin

Admin Dashboard

Platform overview and management

Total Users

7

Total Farmers

3

Total Buyers

3

Total Products

6

Pending Products

0

Total Orders

2

Active Sessions

0

System Status

Quick Actions

Manage Users

Approve Products

Edit Products

Analytics

Recent Activity

Recent activity logs would appear here in a production system.

9.2 SOURCE CODE

App.py

```
from flask import Flask, render_template, request, redirect,
url_for, flash, session, jsonify
from flask_sqlalchemy import SQLAlchemy
from werkzeug.utils import secure_filename
from datetime import datetime
import os
from functools import wraps

from config import Config
from database import db # Import db from database module

# Import models after db is defined
from models import User, Farmer, Buyer, Product, Order,
OrderItem, Promotion, Feedback

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    # Import models first to ensure they're defined before
    # initializing db
    from models import User, Farmer, Buyer, Product, Order,
    OrderItem, Promotion, Feedback

    # Initialize database
    db.init_app(app)

    return app

app = create_app()
```

Helper function to check if user is logged in

```
def login_required(f):  
    @wraps(f)  
    def decorated_function(*args, **kwargs):  
        if 'user_id' not in session:  
            return redirect(url_for('login'))  
        return f(*args, **kwargs)  
    return decorated_function
```

Helper function to check user role

```
def role_required(allowed_roles):  
    def decorator(f):  
        @wraps(f)  
        def decorated_function(*args, **kwargs):  
            if 'user_id' not in session:  
                return redirect(url_for('login'))  
  
            user = User.query.get(session['user_id'])  
            if not user or user.role not in allowed_roles:  
                flash('Access denied. Insufficient permissions.', 'error')  
                return redirect(url_for('index'))  
  
            return f(*args, **kwargs)  
        return decorated_function  
    return decorator
```

File upload helper

```
def allowed_file(filename):  
    return '.' in filename and \  
        filename.rsplit('.', 1)[1].lower() in  
app.config['ALLOWED_EXTENSIONS']
```

Routes


```

# Debug route to test static image rendering
@app.route('/test-image')
def test_image():
    """Test route to verify static image serving"""
    # Get a product that has an image
    product_with_image =
Product.query.filter(Product.image_path.isnot(None)).first()
    if product_with_image:
        image_filename = product_with_image.image_path
        # Create a simple HTML response with the image
        html_response = f"""
<!DOCTYPE html>
<html>
<head><title>Image Test</title></head>
<body>
    <h1>Static Image Test</h1>
    <p>Testing image: {image_filename}</p>
    
    <p><a href="/">Back to Home</a></p>
</body>
</html>
"""
        return html_response
    else:
        return "<h1>No products with images found</h1><a
href="/">Back to Home</a>"

@app.route('/')
def index():
    # Get featured products (approved products with promotions
or high ratings)

```

```

featured_products =
Product.query.filter_by(status='approved').limit(6).all()

# Get promoted products
promoted_products = []
for product in Product.query.filter_by(status='approved').all():
    if product.promotions:
        active_promo = next((promo for promo in
product.promotions if promo.is_active), None)
        if active_promo:
            promoted_products.append(product)

# Limit to 4 promoted products
promoted_products = promoted_products[:4]

return render_template('index.html',
featured_products=featured_products,
promoted_products=promoted_products)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        user = User.query.filter_by(email=email).first()

        if user and user.check_password(password):
            session['user_id'] = user.id
            session['role'] = user.role

            # Redirect based on user role
            if user.role == 'admin':
                return redirect(url_for('admin_dashboard'))

```

```

        elif user.role == 'farmer':
            return redirect(url_for('farmer_dashboard'))
        elif user.role == 'buyer':
            return redirect(url_for('browse_products'))
        else:
            flash('Invalid user role', 'error')
    else:
        flash('Invalid email or password', 'error')

return render_template('auth/login.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        password = request.form['password']
        role = request.form['role']
        farm_name = request.form.get('farm_name', "")
        location = request.form.get('location', "")
        contact_number = request.form.get('contact_number', "")

        # Check if user already exists
        existing_user = User.query.filter_by(email=email).first()
        if existing_user:
            flash('Email already registered', 'error')
            return render_template('auth/register.html')

        # Create new user
        user = User(name=name, email=email, role=role)
        user.set_password(password)

    try:
        db.session.add(user)

```

```

        db.session.commit()

        # Create role-specific profile
        if role == 'farmer':
            farmer = Farmer(user_id=user.id,
farm_name=farm_name, location=location,
contact_number=contact_number)
            db.session.add(farmer)
        elif role == 'buyer':
            buyer = Buyer(user_id=user.id,
contact_number=contact_number)
            db.session.add(buyer)

        db.session.commit()
        flash('Registration successful! Please log in.', 'success')
        return redirect(url_for('login'))
    except Exception as e:
        db.session.rollback()
        flash('Registration failed. Please try again.', 'error')

    return render_template('auth/register.html')

@app.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out.', 'info')
    return redirect(url_for('index'))

# Farmer routes
@app.route('/farmer/dashboard')
@login_required
@role_required(['farmer'])
def farmer_dashboard():
    user = User.query.get(session['user_id'])

```

```

farmer = Farmer.query.filter_by(user_id=user.id).first()
products = Product.query.filter_by(farmer_id=farmer.id).all()

# Calculate some stats
total_products = len(products)
active_products = len([p for p in products if p.status ==
'approved'])
pending_products = len([p for p in products if p.status ==
'pending'])

return render_template('farmer/dashboard.html',
                        farmer=farmer,
                        products=products,
                        total_products=total_products,
                        active_products=active_products,
                        pending_products=pending_products)

# Error handlers
@app.errorhandler(404)
def not_found_error(error):
    return render_template('errors/404.html'), 404

@app.errorhandler(500)
def internal_error(error):
    db.session.rollback()
    return render_template('errors/500.html'), 500

if __name__ == '__main__':
    app.run(debug=True)

```